Latches and Flip-Flops

Contemporary Digital Electronic Systems — Chapter 10

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Clocks
3. Latches
4. Flip-Flops
5. Describing Operation
6. Practical Aspects
7. Flip-Flop and Clock ICs
8. Summary

Why This Chapter Matters

Every circuit so far was combinational: output $= f$(current input). Counters, state machines and processors must remember the past. This chapter adds a memory element (latch or flip-flop) and a clock that decides when memory may change.

After this chapter, you can:

1. State how combinational and sequential circuits differ.
2. Build an SR latch from cross-coupled NAND or NOR gates and analyse it.
3. Distinguish level-sensitive latches from edge-triggered flip-flops.
4. Draw the characteristic table, equation and timing of D, SR, JK and T devices.
5. State setup and hold requirements and find the maximum clock frequency.

© Muním Zabidi

An Opening Question

What happens if a flip-flop's data input changes exactly at the sampling clock edge?

The answer is one of the few places where real hardware admits uncertainty. We return to it with setup and hold times.

© Muním Zabidi

Sequential Circuit = Logic + State

- A sequential circuit is combinational logic plus a bank of flip-flops that store the state.
- On each active clock edge the flip-flops sample their inputs and update the state. Between edges, the logic computes the next state and the outputs.

	Combinational	Latch	Flip-flop
Depends on	Current input only	Current input (when enabled) and stored value	Stored value, sampled at the clock edge
Stores?	No	Yes, while enable is active	Yes, until the next active edge
Responds to	Any input change (after t_{pd})	Input change *while* enable is high (level-sensitive)	Input value *at* the active edge (edge-triggered)

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Clocks

3 Latches

4 Flip-Flops

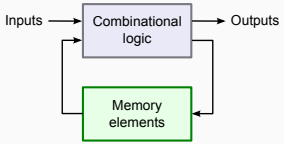
5 Describing Operation

6 Practical Aspects

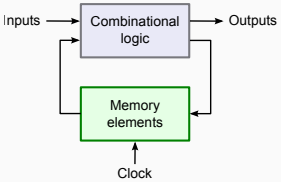
7 Flip-Flop and Clock ICs

8 Summary

Synchronous and Asynchronous Circuits



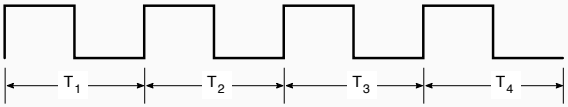
Asynchronous



Synchronous

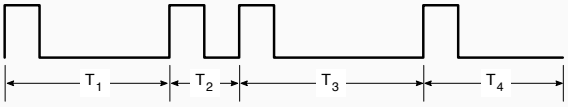
- Synchronous: a clock coordinates all actions: precise timing, reliable, easy to use. Costs clock-distribution power; speed is limited by the slowest path.
- Asynchronous: responds to input changes without a clock; less predictable.

Clock Signals



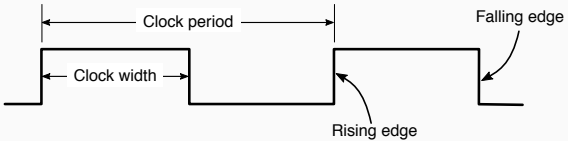
$T_1 = T_2 = T_3 = T_4$

A clock signal



$T_1 \neq T_2 \neq T_3 \neq T_4$

Not a clock signal



Parts of a clock signal

- A clock is a regular pulse train, measured in Hz (GHz in modern circuits).
- Most circuits are edge-triggered: positive-edge (low-to-high) or negative-edge (high-to-low).
- Latches are level-triggered; flip-flops are edge-triggered.

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Clocks

3 **Latches**

4 Flip-Flops

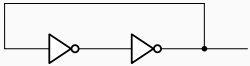
5 Describing Operation

6 Practical Aspects

7 Flip-Flop and Clock ICs

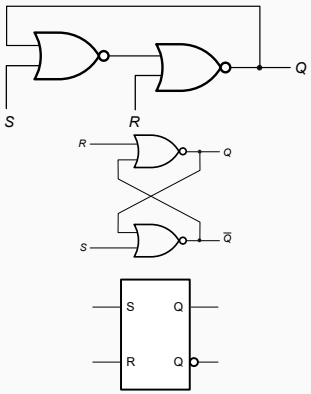
8 Summary

Bistable Circuit



- Two inverters in a loop have two stable states, so they remember a bit (the basis of SRAM).
- The state is random at power-up and cannot be set. Replace the inverters with control gates (NOR) to set and reset it.

SR Latch (NOR)



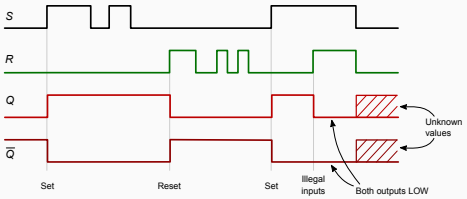
- $S = 1, R = 0$: $Q = 1$ (set). $S = 0, R = 1$: $Q = 0$ (reset). $S = R = 0$: hold.
- $S = R = 1$: both outputs low (forbidden). Releasing both together gives an unpredictable Q: avoid.

S	R	Q^+	$\overline{Q^+}$	Action
0	0	Q	$\overline{Q}$	No change
0	1	0	1	Reset
1	0	1	0	Set
1	1	0	0	Forbidden

SR Latch: Characteristic Table and Timing

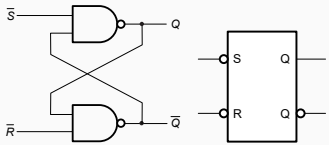
S	R	Q	$\overline{Q}$	Q^+	$\overline{Q^+}$
0	0	0	1	0	1
0	0	1	0	1	0
0	1	0	1	0	1
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	0	1	0
1	1	0	1	0	0
1	1	1	0	0	0

Q = present state; Q^+ = next state.



- $\overline{Q}$ is the opposite of Q except when $S = R = 1$.

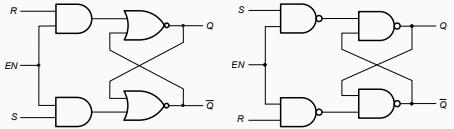
$\overline{S}\,\overline{R}$ Latch (NAND)



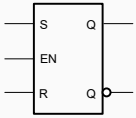
$\overline{S}$	$\overline{R}$	Q^+	$\overline{Q^+}$	Action
0	0	1	1	Forbidden
0	1	1	0	Set
1	0	0	1	Reset
1	1	Q	$\overline{Q}$	No change

- Set and reset are active low.
- More widely used than the NOR latch: NAND gates are usually faster.

Gated SR Latch



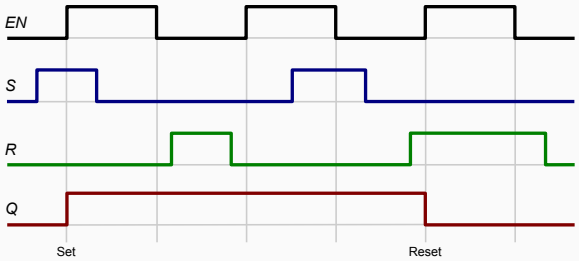
NOR-based NAND-based



- An enable (*EN*) controls a gate in front of *S* and *R*. With $EN = 0$ the latch holds, whatever *S* and *R* do.
- With $EN = 1$ the inputs act as in the plain SR latch. Do not apply $S = R = 1$.

EN	*S*	*R*	Q^+	Action
0	X	X	*Q*	No change
1	0	0	*Q*	No change
1	0	1	0	Reset
1	1	0	1	Set
1	1	1	X	Forbidden

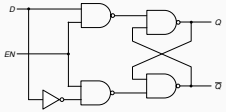
Gated SR Latch: Timing



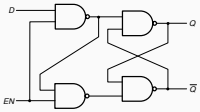
- Output changes only while *EN* is high.
- Both inputs high just before *EN* falls leaves the latch in a forbidden state.

© Muním Zabidi

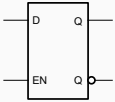
D Latch



Gated SR latch plus an inverter



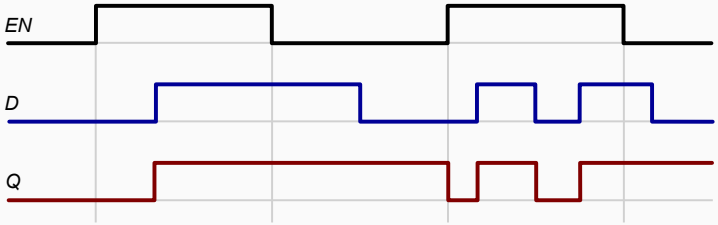
Improved version



- Inputs: enable *EN* and data *D*.
- *EN* = 1: *Q* follows *D* (“transparent”). *EN* = 0: *Q* holds the value *D* had when *EN* fell.

EN	*D*	Q^+	Action
0	X	*Q*	Storage
1	0	0	Transparent
1	1	1	Transparent

D Latch: Timing



- While *EN* is high the output tracks *D*; when *EN* falls the output freezes.

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Clocks

3 Latches

4 Flip-Flops

5 Describing Operation

6 Practical Aspects

7 Flip-Flop and Clock ICs

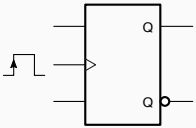
8 Summary

Edge-Triggered Flip-Flops

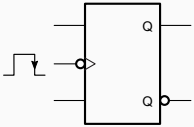
- Flip-flops respond to clock edges; latches respond to clock levels.
- PET: rising edge. NET: falling edge.
- In symbols the clock input is a triangle; a bubble marks negative-edge sensitivity.

Clock edge $\neq$ clock level

A flip-flop updates at one instant. A latch is transparent for the whole active level. Confusing them is a common source of design bugs.

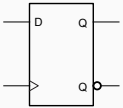


PET



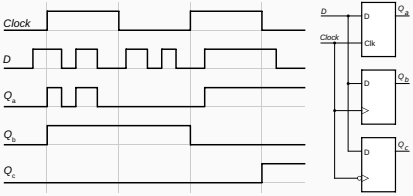
NET

© Muním Zabidi



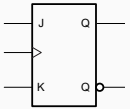
Clk	*D*	Q^+	Action
0	X	*Q*	No change
1	X	*Q*	No change
↑	0	0	Reset
↑	1	1	Set

- Stores one bit or delays a signal by one clock cycle.
- Unlike the D latch it has no transparent mode: it copies *D* to *Q* at the clock edge, and *Q* is then unchanged until the next edge.



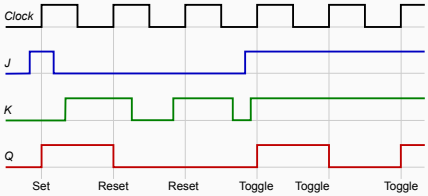
© Muním Zabidi

JK Flip-Flop



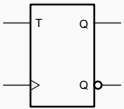
Clk	*J*	*K*	Q^+	Action
0/1	X	X	Q	No change
↑	0	0	Q	No change
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	$\overline{Q}$	Toggle

- Versatile: set, reset, hold and toggle.
- It removes the ambiguity of the SR latch: $J = K = 1$ is allowed and toggles.



© Muním Zabidi

T Flip-Flop



Clk	*T*	Q^+	Action
0/1	X	Q	No change
↑	0	Q	Hold
↑	1	$\overline{Q}$	Toggle

- $T = 0$: the clock has no effect. $T = 1$: the output toggles on each active edge.

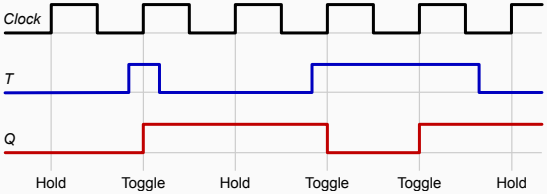


Table of Contents

1 Why This Chapter

2 Clocks

3 Latches

4 Flip-Flops

5 **Describing Operation**

6 Practical Aspects

7 Flip-Flop and Clock ICs

8 Summary

Describing a Flip-Flop

- The characteristic table shows the next state Q^+ for each present state and input. Three descriptions follow from it:
- Characteristic equation: Q^+ as a function of Q and the inputs.
- Excitation table: the inputs needed to cause a desired transition $Q \to Q^+$.
- State diagram: circles for the two states, arrows for transitions.

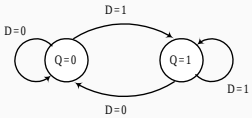
Characteristic equations

D: $Q^+ = D$ T: $Q^+ = T \oplus Q$
JK: $Q^+ = J\overline{Q} + \overline{K}Q$ SR: $Q^+ = S + \overline{R}Q$, with $SR = 0$

D and T: Excitation Tables and State Diagrams

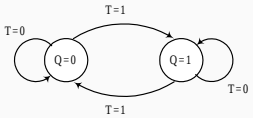
D flip-flop

Q	Q^+	D
0	0	0
0	1	1
1	0	0
1	1	1



T flip-flop

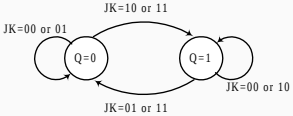
Q	Q^+	T
0	0	0
0	1	1
1	0	1
1	1	0



JK and SR: Excitation Tables and State Diagrams

JK flip-flop

Q	Q^+	J	K
0	0	0	×
0	1	1	×
1	0	×	1
1	1	×	0



SR latch

Q	Q^+	S	R
0	0	0	×
0	1	1	0
1	0	0	1
1	1	×	0

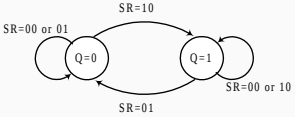


Table of Contents

1 Why This Chapter

2 Clocks

3 Latches

4 Flip-Flops

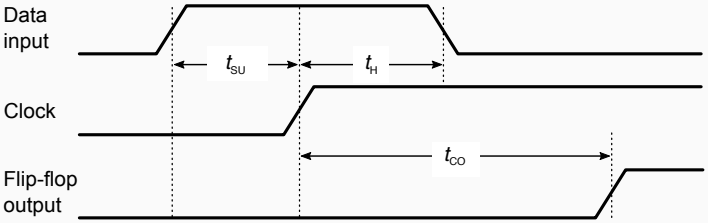
5 Describing Operation

6 **Practical Aspects**

7 Flip-Flop and Clock ICs

8 Summary

Setup and Hold Times



- Setup time t_{SU}: the input must be stable this long *before* the clock edge.
- Hold time t_H: the input must stay stable this long *after* the edge.
- Clock-to-output delay t_{CO}: time from the clock edge to a change at the output.
- If setup or hold is not met, the flip-flop may not capture the data reliably.

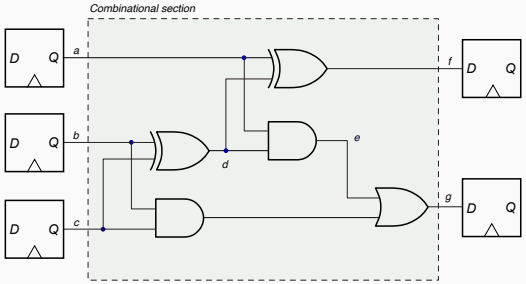
© Munim Zabidi

Maximum Clock Frequency

$$f_{\max} = \frac{1}{t_{CO} + t_{CPD} + t_{SU}}$$

- AND, OR and XOR delays: 10, 11 and 16 ns. $t_{CO} = 10$ ns, $t_{SU} = 3$ ns.
- Critical path: XOR, then AND, then OR.

$t_{CPD} = 16 + 10 + 11 = 37$ ns
$f_{\max} = 1/(10 + 37 + 3)$ ns $= 20$ MHz

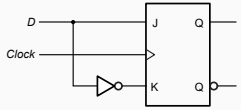


Data changing at the clock edge

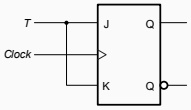
If D changes inside the setup–hold window around the edge, the flip-flop cannot be relied on to capture the old or the new value: the sampled output is unreliable. Designs must keep inputs stable for t_{SU} before and t_H after every active edge.

© Muním Zabidi

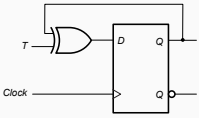
Flip-Flop Conversion



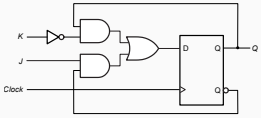
JK to D (inverter between J and K)



JK to T (tie J and K)



D to T ($D = T \oplus Q$)



D to JK

- The JK flip-flop can be turned into any other type.
- Modern designs favour D flip-flops; convert a D flip-flop with the target's characteristic equation:

T: $Q^+ = T \oplus Q$, so feed $T \oplus Q$ to D.
JK: $Q^+ = J\overline{Q} + \overline{K}Q$, so feed that to D.

Comparing Flip-Flops

Q	Q^+	S	R	D	J	K	T
0	0	0	×	0	0	×	0
0	1	1	0	1	1	×	1
1	0	0	1	0	×	1	1
1	1	×	0	1	×	0	0

- New designs: D flip-flops: efficient, easy to design, supported by ASIC/FPGA tools.
- SSI/MSI: JK: versatile, but more wiring.
- Counters: T: least logic.

Table of Contents

1 Why This Chapter

2 Clocks

3 Latches

4 Flip-Flops

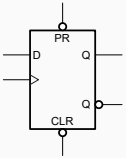
5 Describing Operation

6 Practical Aspects

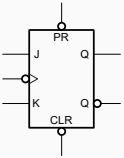
7 **Flip-Flop and Clock ICs**

8 Summary

74x74 and 74x76



74x74: two PET D flip-flops



74x76: two NET JK flip-flops

- Each has asynchronous $\overline{\text{PR}}$ (preset: $Q = 1$) and $\overline{\text{CLR}}$ (clear: $Q = 0$), working regardless of the clock.
- With both inactive, the device captures D (or J, K) on the active edge.
- These SSI chips are for labs, but asynchronous preset/clear are still used to initialise circuits.

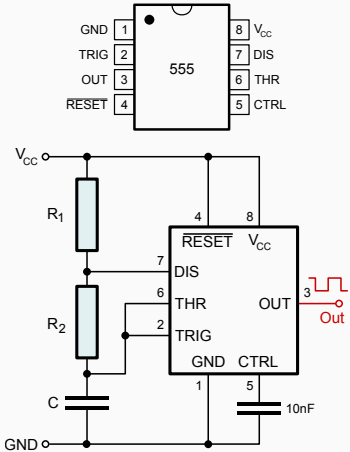
© Muním Zabidi

74x74 Truth Table

Preset	Clear	D	Clock	Q^+
1	1	0	↑	0
1	1	1	↑	1
1	1	X	0	Q
1	1	X	1	Q
0	1	X	X	1
1	0	X	X	0
0	0	X	X	not allowed

© Munim Zabidi

The 555 Timer as a Clock



- A clock has a frequency and a duty cycle (percent of the period spent high).
- In astable mode, R_1, R_2 and C set the timing:

$$f = \frac{1.44}{(R_1 + 2R_2)C}, \quad D = \frac{R_1 + R_2}{R_1 + 2R_2} \times 100\%$$

$$t_{high} = 0.693(R_1 + R_2)C, \quad t_{low} = 0.693R_2C$$

- Cheap, but accuracy depends on component tolerance. Use a crystal oscillator when stability matters.

© Munim Zabidi

Table of Contents

1 Why This Chapter

2 Clocks

3 Latches

4 Flip-Flops

5 Describing Operation

6 Practical Aspects

7 Flip-Flop and Clock ICs

8 **Summary**

Summary

- A sequential circuit has state; a combinational circuit does not.
- A latch is level-sensitive (transparent while enabled). A flip-flop is edge-triggered.
- SR, JK, D and T differ only in their characteristic equation; one type can always be built from another.
- Setup and hold times define the window around the clock edge in which data must be stable. Violating either may produce an unpredictable output.
- $f_{\max}$ follows from $t_{CO} + t_{CPD} + t_{SU} \leq T_{clk}$.

Looking ahead

Chapter 11 wires flip-flops into registers: parallel groups that store and shift multi-bit values.